

FA26: DATA-266 Sec 21 & 71 - Generative AI and LLM

Homework 5

Siva Surya Chandran
019130215

SID4	SEED	SLICE	HP_ID	CLS_A	CLS_B
3215	3215	215	5	5	2

Note on SID4. My SJSU ID is 019130215, so the last four digits are 0215. Taken literally that has a leading zero, which collapses to the 3-digit number 215 when used as a number and makes SID4 ambiguous with SLICE. I used 3215 instead - the last four digits with the preceding digit 3 in place of the leading zero - so that SID4 is a genuine 4-digit value and SEED/SLICE stay distinct. All parameters are derived from SID4 = 3215. HW5 uses a fixed sweep ($r=4$ vs $r=16$), so HP_ID isn't used, and SLICE/CLS_A/CLS_B aren't used either (full DialogSum splits, no classes). SEED = 3215 is used for every random seed.

What I did

For this assignment I fine-tuned google/flan-t5-base (about 248M parameters) to summarize conversations, using LoRA through HuggingFace's PEFT library, on the DialogSum dataset (neil-code/dialogsum-test). I ran the model on the same 2 test dialogues before and after fine-tuning to see what changed, scored both on the whole test set with ROUGE, and at the end trained a second adapter with a smaller rank ($r=4$ vs $r=16$) to see how much the rank actually matters. Everything below - the code, all the outputs, and the write-up for each part - is in the notebook HW5.ipynb, which I ran top to bottom on a Google Colab T4 GPU with SEED = 3215.

The assignment says flan-t5-small "or similar". I went with flan-t5-base because small's summaries were too weak to show a meaningful difference before and after.

Getting the data ready

The dataset has 1,999 train, 499 validation and 499 test rows. I turned every row into an input/target pair: the input is the dialogue with "Summarize the following conversation." in front (FLAN-T5 was instruction-tuned, so it works better when you tell it the task), and the target is the human-written summary. Inputs are cut off at 512 tokens and summaries at 128. The median input is 213 tokens and only 45 of the 1,999 training dialogues are long enough to get cut.

One thing I want to flag: the test split has 499 rows but only 167 different dialogues. Every dialogue shows up 3 times (test_0_1, test_0_2, test_0_3), once per human summary. The first time I picked my 2 demo dialogues I accidentally picked the same conversation twice. So I grouped the test rows by dialogue, and every test dialogue is used once and scored against its 3 reference summaries.

LoRA setup

LoRA keeps all the original weights frozen and learns a small low-rank update for each chosen weight matrix, $\Delta W = (\alpha/r) \cdot BA$. Only A and B get trained, which is a tiny fraction of the model. I put the adapters on the q and v attention projections in every attention block (encoder and decoder).

Setting	Value
Base model	google/flan-t5-base
Rank r	16
lora_alpha	32 (alpha/r = 2)
lora_dropout	0.05
Target modules	q, v
Total parameters	249,347,328
Trainable parameters	1,769,472 (0.71%)
Saved adapter size	7.1 MB

Training

I wrote a plain PyTorch training loop on the full training split: AdamW with learning rate $1e-3$ (LoRA usually wants a higher learning rate than full fine-tuning since it updates so few weights), 5% warmup then linear decay, gradient clipping at 1.0, and an effective batch size of 8 (micro-batches of 4 with 2 gradient-accumulation steps). 3 epochs is 750 optimizer steps. I trained in fp32 because T5 is known to blow up to NaN in fp16 and the T4 doesn't support bf16. Each training run took about 11 minutes.

I first tried running it on my Mac (16 GB), but training used up all the memory and slowed to a crawl, which is why I moved to Colab.

	Before	Epoch 1	Epoch 2	Epoch 3
Validation loss	1.83	1.11	1.08	1.08

Training loss drops fast in the first ~75 steps (1.76 to 1.34) and after that slowly goes down with a lot of step-to-step noise, ending at 1.10. Validation loss was still going down at epoch 3, so it wasn't overfitting yet.

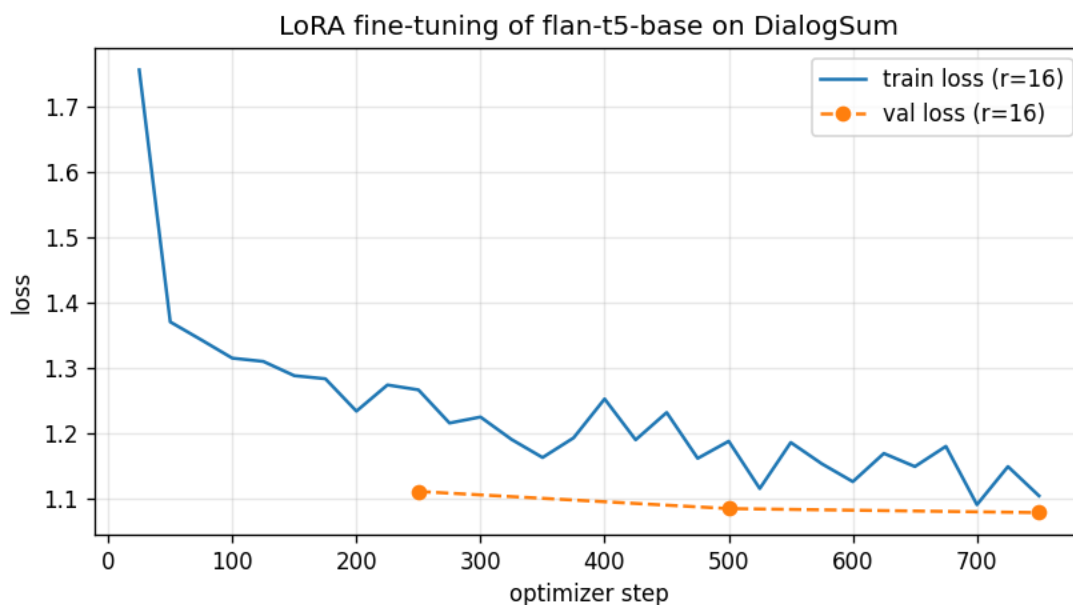


Figure 1: training and validation loss for LoRA $r=16$

Before vs after fine-tuning

Same 2 test dialogues (test_0 and test_1), same greedy decoding, before and after. The model never saw these during training.

test_0 - #Person1# dictates a memo to Ms. Dawson banning Instant Messaging at work:

Reference: Ms. Dawson helps #Person1# to write a memo to inform every employee that they have to change the communication method and should not use Instant Messaging anymore.

Before: The memo is to be distributed to all employees by this afternoon.

After (r=16): #Person1# asks Ms. Dawson to take a dictation for #Person1#. Ms. Dawson tells #Person1# the new policy applies to all communications, including intra-office communications, and will be placed on probation.

test_1 - #Person2# is late because of traffic and gets talked into taking public transport:

Reference: #Person2# arrives late because of traffic jam. #Person1# persuades #Person2# to use public transportations to keep healthy and to protect the environment.

Before: The traffic jam at the Carrefour intersection has caused a lot of congestion.

After (r=16): #Person2# got stuck in traffic again. #Person1# suggests taking public transport to work. #Person2# thinks it's better for the environment and biking to work will give #Person2# more freedom.

Before fine-tuning, the model basically picks one detail from the conversation (the memo deadline, the traffic jam) and writes a single short sentence about it, missing the actual point. After fine-tuning, the summaries read like the DialogSum ones: they refer to people as #Person1#/#Person2#, and go through the whole conversation. For test_1 it now covers the traffic, the public transport suggestion, the environment and biking to work.

It's still not perfect. In test_0 it mixes up who's doing what: it says Ms. Dawson tells #Person1# the new policy, when it's really #Person1# dictating the memo to her. It never says the policy bans Instant

Messaging, and "will be placed on probation" is garbled (in the dialogue it's employees who keep using IM who get probation). In test_1 it stops before the ending, where #Person2# decides to quit driving.

Two examples don't say much on their own, so I also scored both models with ROUGE on all 167 test dialogues (each prediction compared against the best of its 3 references):

Model	ROUGE-1	ROUGE-2	ROUGE-L	Avg words
flan-t5-base, no fine-tuning	28.38	11.06	24.92	15.5
LoRA r=16	49.06	24.17	41.25	23.8
Human references	-	-	-	19.3

ROUGE-1 went from 28.4 to 49.1, and the summaries got longer (15.5 to 23.8 words), a bit longer than the humans' 19.3.

Experiment: LoRA rank r=4 vs r=16

For my experiment I trained a second adapter with r=4. Everything else is the same as the r=16 run: same data, seed, learning rate, epochs, batch size and target modules. The one thing I was careful about is alpha: I kept alpha/r fixed at 2, so r=4 gets alpha=8. If I left alpha at 32 for both, the r=4 updates would get scaled 4x more than the r=16 ones, and I'd be changing the effective learning rate along with the rank.

	r=4	r=16
lora_alpha	8	32
Trainable params	442,368 (0.18%)	1,769,472 (0.71%)
Adapter file size	1.8 MB	7.1 MB
Final train loss	1.1619	1.1038
Final val loss	1.0948	1.0781
ROUGE-1 / ROUGE-2 / ROUGE-L	48.84 / 23.73 / 41.40	49.06 / 24.17 / 41.25
Avg summary length	21.8 words	23.8 words
Training time (T4)	667 s	668 s

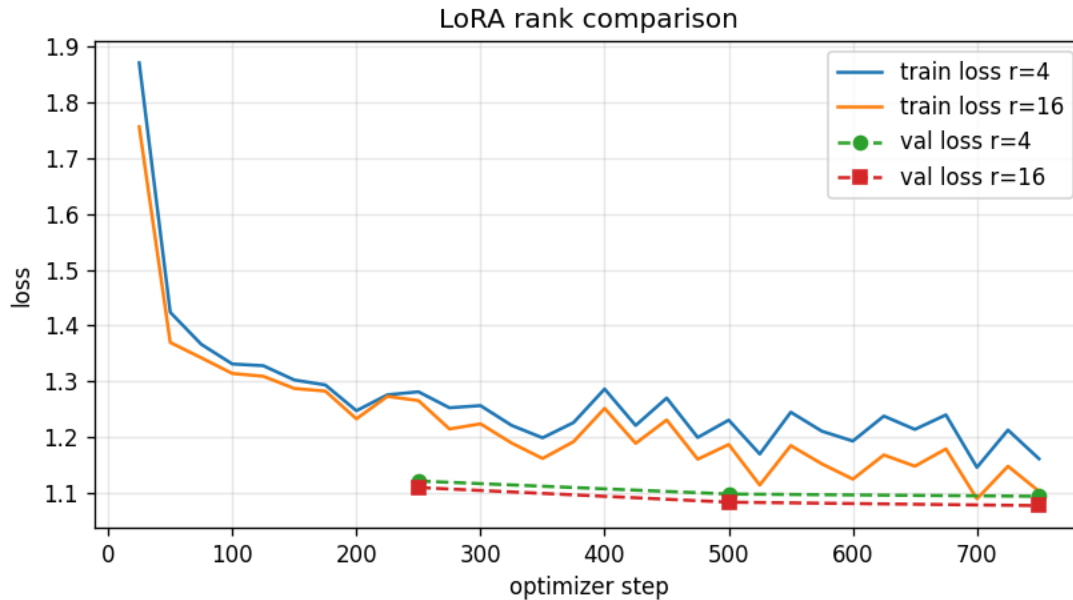


Figure 2: loss curves for r=4 vs r=16

On ROUGE, r=4 and r=16 are basically tied. r=16 is ahead by 0.2 on ROUGE-1 and 0.4 on ROUGE-2, and r=4 is actually ahead on ROUGE-L by 0.15. So r=4 gets the whole jump from the base model (ROUGE-1 28.4 to 48.8) with 4x fewer trainable parameters. The loss does favor r=16: its train and val loss are a little lower at every point, and its curve sits just under the r=4 one the whole way. That lower loss just doesn't show up as better summaries. Training time was the same, because most of the compute goes into running the frozen base model.

When I read the actual summaries, I couldn't say either one is better. The two give different summaries on 145 of the 167 test dialogues, but a lot of that is wording, and they often start with the same sentence. On test_0, r=4 actually says what the policy is (all communication restricted to email and official memos, including external), while r=16 stays vague - but r=4 gets the opening backwards. On test_1, r=4 adds the pollution detail and r=16 adds the "more freedom" point, and neither gets to the ending where #Person2# decides to quit driving.

test_0, r=4: Ms. Dawson asks #Person1# to take a dictation for #Person1#. #Person1# tells #Person2# that all office communications are restricted to email correspondence and official memos. The memo also restricts external communications.

test_1, r=4: #Person2# got stuck in traffic again. #Person1# suggests #Person2# start taking public transport system to work. #Person2# feels bad about the pollution problem in this city. #Person1# suggests biking to work.

Both ranks make the same kind of mistake, and that's mixing up who said what:

- test_2: #Person1# tells Kate about the divorce, but r=4 says "Kate tells #Person2#" and r=16 says "Kate tells #Person1#".
- test_3: #Person1# asks Brian to dance, but both models say Brian asks/invites #Person2# - who is Brian himself.

Going up in rank didn't fix this, so my guess is it's more about the base model's size than the adapter's capacity.

One caveat: this is one seed and 167 test dialogues. When I first ran the notebook with seed 42 (before switching to SEED = 3215), r=16 was ahead by about 1 ROUGE point on every metric. With SEED = 3215 the gap is 0.2 and ROUGE-L flips to r=4, so the difference between the ranks is within seed-to-seed noise. With val loss still going down at epoch 3, I think training longer would help more than raising the rank.

Quick summary

Model	Trainable params	ROUGE-1	ROUGE-L	Summary quality
No fine-tuning	-	28.38	24.92	One detail, misses the point
LoRA r=4	442,368	48.84	41.40	DialogSum style, mixes up speakers
LoRA r=16	1,769,472	49.06	41.25	DialogSum style, mixes up speakers